

HTL Maturatrainer

DIPLOMARBEIT

verfasst im Rahmen der

Reife- und Diplomprüfung

an der

Höheren Abteilung für Medientechnik

Eingereicht von:

Thomas Gossenreiter
Stefanie Steinmair

Betreuer:

Birgit Schröder

Leonding, 6. April 2026

Ich erkläre an Eides statt, dass ich die vorliegende Diplomarbeit selbstständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt bzw. die wörtlich oder sinngemäß entnommenen Stellen als solche kenntlich gemacht habe.

Die Arbeit wurde bisher in gleicher oder ähnlicher Weise keiner anderen Prüfungsbehörde vorgelegt und auch noch nicht veröffentlicht.

Die vorliegende Diplomarbeit ist mit dem elektronisch übermittelten Textdokument identisch.

Leonding, 6. April 2026

Thomas Gossenreiter & Stefanie Steinmair

Abstract

Brief summary of our amazing work. In English. This is the only time we have to include a picture within the text. The picture should somehow represent your thesis. This is untypical for scientific work but required by the powers that are. Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.



Zusammenfassung

Zusammenfassung unserer genialen Arbeit. Auf Deutsch. Das ist das einzige Mal, dass eine Grafik in den Textfluss eingebunden wird. Die gewählte Grafik soll irgendwie eure Arbeit repräsentieren. Das ist ungewöhnlich für eine wissenschaftliche Arbeit aber eine Anforderung der Obrigkeit. *Bitte auf keinen Fall mit der Zusammenfassung verwechseln, die den Abschluss der Arbeit bildet!* Suspendisse vel felis.

Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.



Inhaltsverzeichnis

1	Einleitung	1
1.1	Ausgangslage	1
1.2	Motivation und Problemstellung	1
1.3	Zielsetzung	1
1.4	Grundlagen und Analyse	1
1.4.1	Marktanalyse	1
	StudySmarter	1
1.4.2	Anforderungen eines digitales Maturatraining	1
1.4.3	Nutzen für Schüler:innen	1
1.5	Lernstrategien	1
1.5.1	Leitner-System	1
1.5.2	Spaced Repetition	2
1.5.3	Vergleich verschiedener Lernstrategien und Schlussfolgerung . .	2
2	Technologische Entscheidungen und Architektur	3
2.1	Backend	3
2.1.1	Datenbankstruktur	3
2.1.2	Quarkus	3
2.2	Frontend – Angular	3
2.2.1	Aufbau und Komponenten	3
2.3	Entwicklungsumgebung und Versionsverwaltung	4
3	Praktische Umsetzung	5
3.1	Usecases	5
3.1.1	Üben	5
3.1.2	Prüfungssimulationen	5
3.1.3	Fortschrittsübersicht	5

3.2	Benutzeroberfläche und Design	5
3.2.1	Mockups und User Flow	5
3.3	Use Case: Üben	5
3.3.1	Fragenhandling und Fragetypen	6
3.3.2	Übungsmodus mit direktem Feedback	8
3.3.3	Lösungswege und Nutzerinteraktion	8
3.3.4	Verwaltung von Fragenpools	8
3.4	Use Case: Prüfungssimulation	8
3.4.1	Prüfungsmodus mit Zeitlimit	8
3.4.2	Bewertung und Ergebnisausgabe	9
3.5	Use Case: Fortschritt einsehen & analysieren	9
3.5.1	Lernstrategien und Fehleranalyse	9
3.5.2	Spaced Repetition nach Leitner	9
3.5.3	Lernstatistiken und Visualisierung	10
3.6	Usability-Test mit Mitschülern	10
Abbildungsverzeichnis		VI
Tabellenverzeichnis		VII
Quellcodeverzeichnis		VIII
Anhang		IX

1 Einleitung

1.1 Ausgangslage

Beschreibung der aktuellen Situation: Warum besteht Bedarf für die Matura-Lernplattform? Probleme beim Lernen, fehlendes Feedback, etc.

1.2 Motivation und Problemstellung

Warum ist das Thema interessant und relevant? Hauptproblem, das gelöst werden soll.

1.3 Zielsetzung

Was soll am Ende entstehen? Funktionen und Ziele der Plattform.

1.4 Grundlagen und Analyse

1.4.1 Marktanalyse

Analyse bestehender Lernplattformen und digitaler Angebote.

StudySmarter

Funktionsweise, Zielgruppe, Vorteile/Schwächen, Vergleich mit eigenem Projekt.

1.4.2 Anforderungen eines digitales Maturatraining

Funktionale und nicht-funktionale Anforderungen.

1.4.3 Nutzen für Schüler:innen

Mehrwert der Plattform: Motivation, individuelles Lernen, Feedback, Vorbereitung.

1.5 Lernstrategien

1.5.1 Leitner-System

Karteikarten-Prinzip, Vorteile, Umsetzung im Projekt.

1.5.2 Spaced Repetition

Verteiltes Lernen, Effektivität, Umsetzung digital.

1.5.3 Vergleich verschiedener Lernstrategien und Schlussfolgerung

Leitner, Spaced Repetition, klassisches Lernen – Vor- und Nachteile - Ergebnis.

2 Technologische Entscheidungen und Architektur

2.1 Backend

Backend-Aufbau

2.1.1 Datenbankstruktur

Datenbanktabellen

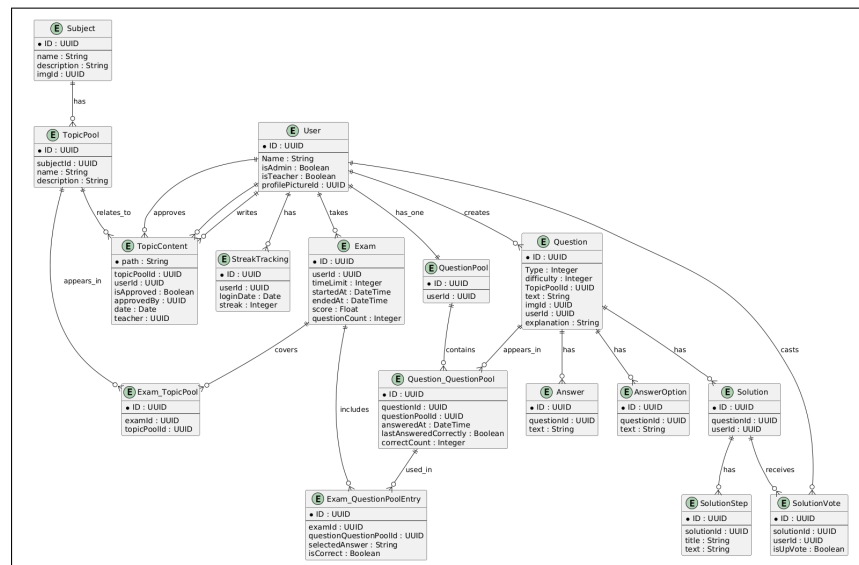


Abbildung 1: Datenbankmodell der Anwendung

2.1.2 Quarkus

REST-Endpunkte, Speicherung von Feedback/Statistiken.

2.2 Frontend – Angular

2.2.1 Aufbau und Komponenten

Struktur der App: Login, Dashboard, Übungsmodus, Prüfung, Ergebnisübersicht. Komponenteninteraktion.

2.3 Entwicklungsumgebung und Versionsverwaltung

Verwendete Tools (IntelliJ, GitHub, Docker), Branching-Strategie, Teamwork.

3 Praktische Umsetzung

3.1 Usecases

3.1.1 Üben

User (Schüler) möchte für gezielte Themen für die Matura lernen. Usecase-Diagramme für User (Schüler) → Fragen beantworten, Feedback erhalten, eigenen Fragenpool erstellen

3.1.2 Prüfungssimulationen

User (Schüler) möchte eine Prüfungssimulation starten, um sein Wissen zu testen. Usecase-Diagramme für User (Schüler) → Fragen beantworten, Feedback am Ende erhalten, Note erhalten

3.1.3 Fortschrittsübersicht

User (Schüler) möchte seinen Lernfortschritt jederzeit einsehen und Statistiken betrachten. Usecase-Diagramme für User (Schüler) → Fortschritt (% der unbeantworteten, falsch beantworteten, 1x, 2x, genug beantworteten Fragen), Durchschnittswertung bei Prüfungen usw.

3.2 Benutzeroberfläche und Design

3.2.1 Mockups und User Flow

UI-Entwürfe (Figma), Nutzerführung Start → Übungsmodus → Ergebnis. Fokus auf Usability.

3.3 Use Case: Üben

Der Kern der Anwendung ist der Übungsmodus, welcher Schüler:innen ermöglicht, gezielt Maturafragen zu trainieren und ihr Wissen Stück für Stück auszubauen. Anders als beim Prüfungsmodus steht hier nicht der Leistungsdruck im Vordergrund, sondern das aktive Lernen mit direktem Feedback. Nutzer:innen soll es ermöglicht werden, ein Fundament für ihr Wissen festigen zu können.

Funktionen umfassen das Auswählen von bestimmten Themen, das Zusammenstellen von eigenen Fragenpools und das Erstellen und Einsehen von Lösungswegen für die jeweiligen Aufgaben. Nutzer:innen können hierbei also genau die Themen lernen, die ihnen Probleme bereiten, und das durch eigene Fragen oder Fragen anderer Nutzer:innen.

Die Plattform unterstützt dabei verschiedene Fragetypen, um das Lernerlebnis flexibler zu gestalten. Um nicht einfach nur ein Ergebnis von "richtig" und "falsch" zu liefern, können Nutzer:innen auch Lösungswege einsehen, um das Verständnis nachhaltig zu fördern.

Durch diese wiederholbare und flexible Übungsumgebung eignet sich die Anwendung sowohl für den schnellen Gebrauch als Wissenscheck als auch für die langfristige Vorbereitung auf Tests, Schularbeiten oder die Matura.

3.3.1 Fragenhandling und Fragetypen

Speicherung, Abruf, Darstellung verschiedener Fragetypen, Feedbacksystem.

Wie in Abbildung 3 dargestellt befindet sich der/die Nutzer:in nun beim Question-Runner, also der Ansicht, bei der Fragen beantwortet und ausgewertet werden können. Hierbei gibt es 3 verschiedene Modi, in der sich der Question-Runner befinden kann:

- **Übungsmodus** (3.3.2): Nutzer:in erhält direkt Feedback zu den Antworten, ohne Zeitdruck.
- **Prüfungsmodus** (3.4.1): Prüfung wird unter Zeitlimit simuliert, Feedback erfolgt erst am Ende.
- **Reviewmodus** (3.4.2): Bereits abgeschlossene Tests können erneut durchgesehen werden, Antworten und Lösungswege sind sichtbar, Änderungen sind nicht möglich.

User-Interface des Question Runners

Üben > Fragen beantworten

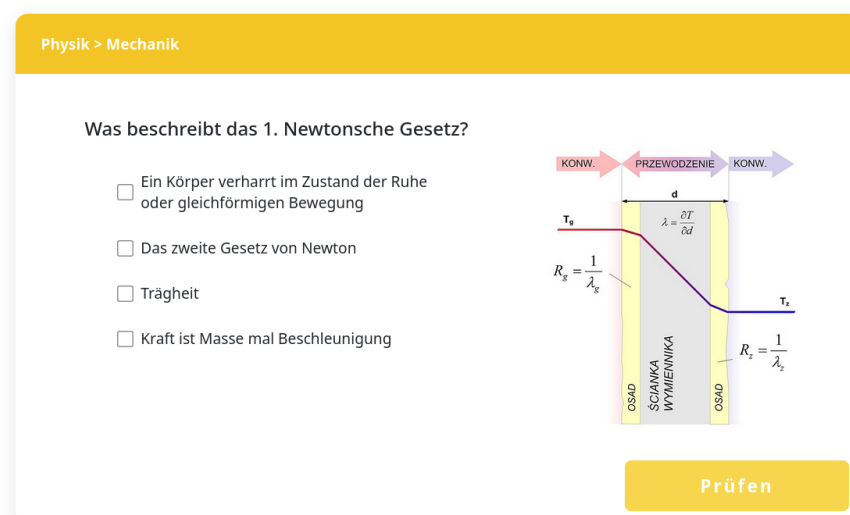


Abbildung 2: Beispielhafte Ansicht einer Frage im Question Runner

Der Question Runner zeigt zunächst, aus welchem Fach und welchem Themenpool die Frage stammt, die der/die Nutzer:in gerade bearbeitet. Direkt darunter wird die Frage als Überschrift angezeigt, rechts daneben ggf. das zugehörige Bild.

Unter der Frage befinden sich die Antwortfelder, abhängig vom Fragetyp: Multiple-Choice-Fragen als Auswahlfelder (Abbildung 3a), Freitextfragen als Eingabefeld (Abbildung 3b). Ganz unten befinden sich die Aktionsbuttons: im Übungsmodus der „**Prüfen**“-**Button**, in den anderen Modi allgemeine Navigationsbuttons (**Weiter/Zurück**).

Datenfluss im Question Runner

Im Übungsmodus ruft der Question Runner zunächst die angegebene Frage anhand ihrer eindeutigen ID ab, falls der/die Nutzer:in eine konkrete Frage auswählen möchte, wie bei der Verwaltung von Fragenpools (3.3.4) beschrieben.

Wurde keine einzelne Frage ausgewählt, werden zunächst die Filterkriterien für Themen und Fragetypen geprüft. Anschließend werden die passenden Fragen aus dem Fragenpool des/der Nutzers/Nutzerin abgerufen und dem Question Runner übergeben. Auf diese Weise wird sichergestellt, dass sowohl gezieltes Üben einzelner Fragen als auch das Bearbeiten thematisch gefilterter Mengen möglich ist.

Der Question Runner erhält von anderen Komponenten im Frontend eine **Liste von Frage-IDs** oder alternativ eine einzelne ID, die der/die Nutzer:in bearbeiten möchte. Diese IDs repräsentieren die Fragen, die im Anschluss nacheinander abgerufen und angezeigt werden. Die Auswahl der IDs erfolgt zuvor in der Themen- oder Fragenpool-Ansicht und wird hier nicht weiter gefiltert.

Das Backend liefert das vollständige Frageobjekt zurück, das alle für die Anzeige im Question Runner erforderlichen Informationen enthält (siehe Abschnitt „User-Interface des Question Runners“).

Nachdem der/die Nutzer:in eine Antwort eingibt, hängt das Verhalten vom Modus ab:

- **Übungsmodus:** Die Antwort wird direkt ans Backend gesendet und dort ausgewertet. Das Ergebnis wird sofort zurückgesendet und im Question Runner angezeigt. Danach ruft der Question Runner automatisch die nächste ID auf.
- **Prüfungsmodus:** Antworten werden gesammelt und erst nach Abschluss der Prüfung ans Backend gesendet. Feedback erfolgt danach.
- **Reviewmodus:** Nur Anzeige von Antworten und Lösungswegen. Der/Die Nutzer:in kann hier also keine Antworten eingeben.

Unterstützung verschiedener Fragetypen

Neben der Unterscheidung der Modi werden im Question Runner die verschiedenen Fragetypen berücksichtigt. Aktuell werden zwei Haupttypen unterstützt:

- **Multiple-Choice-Fragen:** Nutzer:innen wählen eine oder mehrere richtige Antworten aus einer Liste vorgegebener Auswahlmöglichkeiten aus. Multiple-Choice-Fragen sind besonders geeignet, um Faktenwissen abzufragen oder schnelle Wissenskontrollen durchzuführen. Das System wertet die Antwort automatisch aus

und kann im Übungsmodus direkt Feedback geben. Die Darstellung erfolgt als übersichtliche Auswahlfelder, die klar zwischen ausgewählt und nicht ausgewählt unterscheiden. (siehe Abbildung 3a)

- **Freitext-Fragen:** Nutzer:innen geben ihre Antwort in ein Texteingabefeld ein. Dieser Fragetyp eignet sich für offene Aufgabenstellungen, bei denen die Antwort nicht auf eine vorgegebene Auswahl beschränkt ist. Im Backend wird die Eingabe des/der Nutzer:in mit einer Menge möglicher korrekter Antworten abgeglichen. Entspricht die Eingabe einer dieser Antwortmöglichkeiten, wird die Antwort als korrekt bewertet, andernfalls als falsch. Im Frontend steht ein ausreichend großes Eingabefeld bereit, um längere Antworten komfortabel einzugeben (siehe Abbildung 3b).

Üben > Fragen beantworten

(a) Multiple-Choice-Frage

Üben > Fragen beantworten

(b) Freitext-Frage

Abbildung 3: Question-Runner: Beispiele für verschiedene Fragetypen

3.3.2 Übungsmodus mit direktem Feedback

Technische Umsetzung, Ablauf, User Experience.

3.3.3 Lösungswege und Nutzerinteraktion

Einschauen von Lösungswegen, Feedbackmechanismen

3.3.4 Verwaltung von Fragenpools

Auswahl eigener Fragen, Erstellung von individuellen Pools, Backend- und Frontend-Umsetzung.

3.4 Use Case: Prüfungssimulation

3.4.1 Prüfungsmodus mit Zeitlimit

Simulation der Prüfung, Zeitlimit, realistische Prüfungssituation.

3.4.2 Bewertung und Ergebnisausgabe

Ergebnisanzeige am Ende, Notenberechnung, Auswertung.

3.5 Use Case: Fortschritt einsehen & analysieren

Ein wichtiger Bestandteil unserer Lernplattform ist Fortschrittseinsicht, um den User die Möglichkeit zu bieten seinen aktuellen Wissensstand jeder Zeit einsehen zu können. Mit übersichtlichen Diagrammen und Statistiken hat der User immer einen klaren Überblick auf seinen Lernfortschritt und kann so effizient einschätzen wie viel Übungsbedarf besteht. Filteroptionen ermöglichen es auch nach verschiedenen Themenpools zu filtern damit Schüler gezielt erkennen in welchen Themenpools sie noch mehr Übung bräuchten.

Neben der Darstellung des Lernfortschritts mit Diagrammen sind Statistiken auch zur Motivation gut geeignet, ein sichtbarer Fortschritt motiviert für zukünftiges Lernen. Ein positives Lernerlebnis kann Nutzer auch für weitere Lernziele anspornen bzw. ihnen helfen regelmäßiger zu üben um ihre Leistung zu verbessern.

Aufgebaut sind die Statistiken und Diagramme auf den Ergebnissen und Daten aus dem Übungsmodus oder wenn der User eine Frage explizit übt. Die Auswahl der gestellten Fragen erfolgt über ein recherchiertes und ausgewähltes Lernsystem, um den Wissenstand langfristig im Kopf zu behalten

3.5.1 Lernstrategien und Fehleranalyse

Identifikation von Wiederholungsbedarf und Schwachstellen.

Um den Lernerfolg langfristig zu verbessern, ist es notwendig, das Lernverhalten analysieren und Schwachstellen zu erkennen. Auf Grundlage der Fehleranalyse wird der Wiederholungsbedarf festgelegt. Da Nutzer:innen verstärkt die Fragen oder Themenpools üben die ihnen schwer fallen wird ihr Wissen und Sicherheit gestützt. Aufgaben und Fragen die sie bereits beherrschen und mehrfach korrekt beantwortet hatten, kommen hingegen in selteneren Abständen bzw. fallen zur gänze raus. Durch diese Anpassungen wird effizienter gelernt und somit auch mehr Lernerfolg erzielt.

Darüber hinaus unterstützt diese Funktion das selbstreflektierende Lernen. Die Nutzer:innen können nachvollziehen und selbst Muster erkennen, bei welchen Inhalten oder Fragen sie des öfteren mal Schwierigkeiten haben. Dadurch können die User selbst sehen wo sie Probleme haben und besser ihre Personalisierten Fragenpools auswählen, diese können sie anschließend üben.

3.5.2 Spaced Repetition nach Leitner

Technische Umsetzung der Lernstrategie, Speicherintervalle, Fortschrittslevel.

Die Grundlage für die Lernplattform bildet das Leitner-System in kombination mit Spaced Repetition, die in Kapitel 1.5 beschrieben wurden. Die Lernmethoden bestehen aus 3 wichtigen Punkten

1. **Level:** Fragen werden in Level eingeteilt - je öfter richtig desto höher das Level

2. **Wiederholungsintervall:** Fragen werden gesperrt wenn man sie richtig hatte und erst nach einen gewissen Zeitraum entsperrt
3. **Anzeige:** Gesperrte und offene Fragen werden angezeigt Die Anzeige im Frontend dient dazu den

In Abbildung 4 Homekomponente bzw. die Umsetzung der Lernstrategien sieht man die verschiedenen "Level". Die Level sind nach dem Leitner-System aufgebaut, von denen es in diesem Fall vier gibt: Startlevel, Basislevel, Trainingslevel und Highscorelevel.

Jede Frage wird durch QuestionPoolEntry abgebildet, diese speichert alle relevante Daten für die Fehleranalyse oder das Wiederholen. Für das einteilen in die Level sind die Einträge lastAnsweredCorrectly und correctCount wichtig. LastAnsweredCorrectly merkt sich ob die Frage das letzte mal korrekt beantwortet wurden und correctCount wie oft du diese Frage bereits richtig hattest. Mithilfe einer Abfrage kann eingeordnet werden welche Frage in welches Level kommt. Es verläuft nach dem Schema

- **Startlevel:** Dort sind die falsch oder nicht beantworteten Fragen, diese sind immer zum üben offen.
- **Basislevel:** Hier befinden sich Fragen die einmalig richtig beantwortet waren, direkt danach sind sie für 1 Tag gesperrt.
- **Trainingslevel:** Fragen die der User 2x richtig hatte kommen ins Trainingslevel und sind nun 3 Tage gesperrt.
- **Highscorelevel:** Hier befinden sich Fragen die schon zur Genüge geübt wurden und somit kaum mehr geübt werden müssen.

Das Sperren erfolgt durch die Auswertung des Parameters answeredAt. So lässt sich nachvollziehen, wann der Nutzer die Frage zuletzt bearbeitet hat, ob die Antwort korrekt oder falsch war, in welchem Level sich die Frage zuvor befand bzw. wie hoch der correctCount ist. Schlussendlich lässt sich dann sagen wie lange die Frist zum Wiederholen eingestellt wird.

Mit einen klick auf üben werden nicht nur irgendwelche Fragen ausgewählt, sondern genau die die der User am dringendsten benötigt. Sozusagen die falsch oder nicht beantworteten Fragen zuerst, darauffolgend erst die mehrmals korrekt beantworteten Inhalte.

Die Anzeige im Frontend dient dazu den Nutzer:innen einen Überblick über ihren eigenen Lernfortschritt zu geben. Die einzelnen Level im Leitner-System werden als Progressbars visualisiert und mit Farben wird dem User sein Fortschritt symbolisiert.

3.5.3 Lernstatistiken und Visualisierung

Erhebung und grafische Darstellung der Daten zur Nutzerleistung. Bibliotheken und Darstellungskonzepte.

3.6 Usability-Test mit Mitschülern

Testbeschreibung, Szenarien, Feedback, Umsetzung von Verbesserungen.

Abbildungsverzeichnis

1	Datenbankmodell der Anwendung	3
2	Beispielhafte Ansicht einer Frage im Question Runner	6
3	Question-Runner: Beispiele für verschiedene Fragetypen	8

Tabellenverzeichnis

Quellcodeverzeichnis

Anhang